

UniApp showActionSheet：底部操作菜单的实战指南

在UniApp跨平台开发中，`uni.showActionSheet()` 是实现“从底部向上弹出操作菜单”的核心API，适用于提供多个并列操作选项的场景（如“分享到微信/QQ”“编辑/删除”“查看大图/保存图片”等）。其核心优势是原生适配各平台样式（微信小程序、App、H5等），交互流畅且无需手动构建弹窗结构。本文将从基础用法、参数配置、场景实战到避坑技巧，完整覆盖该API的使用要点。

一、核心认知：showActionSheet的特性与适用场景

showActionSheet弹出的菜单从屏幕底部滑入，顶部通常有关闭按钮或“取消”选项，操作项横向排列（部分平台为纵向），选择后菜单自动关闭并触发对应逻辑。其核心特性与适用场景需明确区分，避免与showModal等弹窗混淆。

1. 核心特性

- 弹出方式**：固定从底部向上滑入，自带过渡动画，符合移动端用户操作习惯。
- 操作选项**：支持配置多个操作按钮，默认包含“取消”选项（可隐藏），选择后自动关闭菜单。
- 样式适配**：自动匹配各平台原生样式（如微信小程序菜单背景为白色，App端支持圆角优化）。
- 交互逻辑**：点击操作项或空白区域（部分平台）可关闭菜单，不阻断页面其他事件（关闭后恢复）。
- 限制条件**：操作项数量建议控制在3-6个，过多会导致菜单滚动；不支持自定义图标（需通过文字区分）。

2. 典型适用场景

- 文件操作**：“打开/保存/分享/删除”文件的多选项集合。
- 内容交互**：文章的“收藏/举报/分享到朋友圈”等操作。
- 功能入口**：个人中心的“切换账号/设置/帮助中心”快捷入口。
- 状态切换**：订单的“取消订单/申请退款/联系商家”等操作。

二、基础用法：语法与最简示例

showActionSheet的核心是通过 `itemList` 参数配置操作选项，支持回调函数和Promise两种调用方式，基础配置即可满足大部分简单场景需求。

1. 语法结构

Code block

```
1  // 1. 回调函数形式
2  uni.showActionSheet({
3    itemList: ['选项1', '选项2', '选项3'], // 必选：操作选项列表
4    itemColor: '#1890FF', // 可选：操作项文字颜色
5    cancelText: '取消', // 可选：取消按钮文字
6    cancelColor: '#666666', // 可选：取消按钮文字颜色
7    success: (res) => { // 可选：用户选择操作后的回调
8      console.log('选中的索引: ', res.tapIndex) // tapIndex为选中项的下标（从0开始）
9    },
10   fail: (err) => { // 可选：调用失败或用户取消的回调
11     if (err.errMsg === 'showActionSheet:fail cancel') {
12       console.log('用户点击取消')
13     } else {
14       console.error('菜单调用失败: ', err)
15     }
16   },
17   complete: () => { // 可选：无论成功失败都执行的回调
18     console.log('菜单操作完成')
19   }
20 })
21
22 // 2. Promise形式（推荐，适配异步流程）
23 try {
24   const res = await uni.showActionSheet({
25     itemList: ['选项A', '选项B'],
26     itemColor: '#FF4D4F'
27   })
28   console.log('选中索引: ', res.tapIndex)
29 } catch (err) {
30   if (err.errMsg.includes('cancel')) {
31     console.log('用户取消操作')
32   } else {
33     console.error('调用失败: ', err)
34   }
35 }
```

2. 最简示例：基础操作菜单

仅配置必填参数 `itemList`，实现一个包含3个操作项的基础菜单，点击后获取选中索引并执行对应逻辑：

```
1  <template>
2    <view class="demo-container">
3      <image
4        src="/static/images/photo.jpg"
5        mode="widthFix"
6        class="photo"
7        @longpress="showPhotoAction"
8      ></image>
9      <text class="tip">长按图片弹出操作菜单</text>
10    </view>
11  </template>
12
13  <script setup>
14  // 长按图片弹出操作菜单
15  const showPhotoAction = () => {
16    uni.showActionSheet({
17      itemList: ['查看大图', '保存图片', '分享给好友'], // 核心操作选项
18      itemColor: '#1890FF', // 统一操作项颜色
19      success: (res) => {
20        // 根据选中索引执行不同逻辑
21        switch (res.tapIndex) {
22          case 0:
23            console.log('执行查看大图逻辑')
24            // 跳转大图页面: uni.navigateTo({ url: '/pages/big-photo/big-photo' })
25            break
26          case 1:
27            console.log('执行保存图片逻辑')
28            savePhotoToLocal()
29            break
30          case 2:
31            console.log('执行分享逻辑')
32            sharePhoto()
33            break
34        }
35      }
36    })
37  }
38
39  // 模拟保存图片逻辑
40  const savePhotoToLocal = () => {
41    uni.showToast({ title: '图片保存成功', icon: 'success' })
42  }
43
44  // 模拟分享逻辑
45  const sharePhoto = () => {
46    uni.showToast({ title: '分享功能开发中', icon: 'none' })
47  }
```

```
48 </script>
49
50 <style scoped>
51 .demo-container {
52   padding: 30rpx;
53   display: flex;
54   flex-direction: column;
55   align-items: center;
56 }
57 .photo {
58   width: 100%;
59   border-radius: 10rpx;
60   margin-bottom: 20rpx;
61 }
62 .tip {
63   color: #666;
64   font-size: 24rpx;
65 }
66 </style>
```

三、核心参数详解：定制菜单样式与行为

showActionSheet的参数虽不多，但通过灵活配置可适配不同场景，核心参数如下表所示，其中 `itemList` 为唯一必选参数。

1. 完整参数表

参数名	类型	可选值/说明	默认值
itemList	Array<String>	操作选项数组，长度1-9（建议3-6个）	无
itemColor	String	十六进制颜色值（如 #1890FF）	#000000
cancelText	String	取消按钮文字，空字符串可隐藏取消按钮	"取消"
cancelColor	String	十六进制颜色值	#000000
alertText	String	菜单顶部的提示文本（仅微信小程序支持）	无
success	Function	回调参数res包含tapIndex（选中项索引）	无

fail	Function	回调参数err包含错误信息	无
------	----------	---------------	---

2. 关键参数实战

2.1 隐藏取消按钮（特定场景）

当菜单操作是“必选”场景（如订单状态切换），可设置 `cancelText: ""` 隐藏取消按钮，强制用户选择一个操作项：

Code block

```
1 // 订单状态操作菜单（无取消按钮）
2 const showOrderAction = () => {
3   uni.showActionSheet({
4     itemList: ['取消订单', '申请退款', '联系客服'],
5     itemColor: '#FF4D4F', // 红色突出操作风险
6     cancelText: '', // 隐藏取消按钮
7     success: (res) => {
8       const actions = ['取消订单', '申请退款', '联系客服']
9       uni.showToast({ title: `您选择了${actions[res.tapIndex]}`, icon: 'none' })
10    },
11    fail: (err) => {
12      // 无取消按钮时，用户点击空白区域也会触发fail，需处理
13      if (err.errMsg.includes('cancel')) {
14        uni.showToast({ title: '请选择一个操作', icon: 'none' })
15      }
16    }
17  })
18 }
```

2.2 添加顶部提示文本（微信小程序专属）

微信小程序中可通过 `alertText` 在菜单顶部添加提示文本，用于说明操作风险或注意事项：

Code block

```
1 // 包含风险提示的操作菜单（微信小程序）
2 const showRiskAction = () => {
3   uni.showActionSheet({
4     itemList: ['删除账号', '注销数据'],
5     itemColor: '#FF4D4F',
6     alertText: '警告：操作后所有数据将永久删除，不可恢复！', // 顶部提示
7     success: (res) => {
8       // 执行高危操作逻辑（需二次确认）
9       if (res.tapIndex === 0) {
```

```

10     uni.showModal({
11       title: '确认删除',
12       content: '确定要删除账号吗？此操作不可撤销。',
13       success: (modalRes) => {
14         if (modalRes.confirm) {
15           uni.showToast({ title: '账号删除中...', icon: 'loading' })
16         }
17       }
18     })
19   }
20 }
21 })
22 }

```

2.3 自定义按钮颜色（适配项目主题）

根据项目主题色配置操作项和取消按钮的颜色，保持界面风格统一：

Code block

```

1  // 符合项目主题的菜单配置
2  const showThemedAction = () => {
3    uni.showActionSheet({
4      itemList: ['编辑资料', '更换头像', '隐私设置'],
5      itemColor: '#2563EB', // 项目主色
6      cancelText: '返回', // 自定义取消文案
7      cancelColor: '#64748B', // 次要文本色
8      success: (res) => {
9        // 执行对应操作
10      }
11    })
12  }

```

四、实战场景：结合业务逻辑的完整示例

showActionSheet通常需要与其他API（如showLoading、showToast、文件操作API）结合使用，形成完整的业务交互流程，以下是两个典型实战场景。

1. 场景1：图片操作菜单（保存+分享+预览）

长按图片弹出操作菜单，实现“预览大图”“保存到本地”“分享给好友”功能，结合图片相关API完成业务逻辑：

Code block

```
1  <template>
2    <image
3      src="https://example.com/photo.jpg"
4      mode="widthFix"
5      class="pic"
6      @longpress="onPhotoLongPress"
7    ></image>
8  </template>
9
10 <script setup>
11 // 图片地址（实际开发中可能来自接口）
12 const photoUrl = 'https://example.com/photo.jpg'
13
14 // 长按图片触发
15 const onPhotoLongPress = () => {
16   uni.showActionSheet({
17     itemList: ['预览大图', '保存图片', '分享好友'],
18     itemColor: '#1890FF',
19     success: async (res) => {
20       switch (res.tapIndex) {
21         case 0:
22           // 预览大图（调用UniApp预览API）
23           uni.previewImage({
24             urls: [photoUrl], // 预览图片数组
25             current: photoUrl // 当前显示图片
26           })
27           break
28         case 1:
29           // 保存图片到本地（需授权）
30           await savePhoto(photoUrl)
31           break
32         case 2:
33           // 分享图片（模拟微信分享）
34           sharePhoto(photoUrl)
35           break
36       }
37     }
38   })
39 }
40
41 // 保存图片到本地
42 const savePhoto = async (url) => {
43   try {
44     // 1. 检查保存权限
45     const authRes = await uni.getSetting({ scope: 'scope.writePhotosAlbum' })
46     if (!authRes.authSetting['scope.writePhotosAlbum']) {
47       // 2. 无权限，引导授权
```

```

48     const authResult = await uni.authorize({ scope: 'scope.writePhotosAlbum'
    })
49     if (!authResult.authSetting['scope.writePhotosAlbum']) {
50         throw new Error('未获得保存权限')
51     }
52 }
53 // 3. 有权限，保存图片
54 uni.showLoading({ title: '保存中...', mask: true })
55 const saveRes = await uni.saveImageToPhotosAlbum({ filePath: url })
56 uni.hideLoading()
57 uni.showToast({ title: '图片保存成功', icon: 'success' })
58 } catch (err) {
59     uni.hideLoading()
60     if (err.message.includes('权限')) {
61         uni.showToast({ title: '请在设置中开启保存权限', icon: 'none' })
62     } else {
63         uni.showToast({ title: '保存失败，请重试', icon: 'error' })
64     }
65 }
66 }
67
68 // 模拟分享图片
69 const sharePhoto = (url) => {
70     // 实际开发中调用微信分享API或自定义分享组件
71     uni.showModal({
72         title: '分享图片',
73         content: '已唤起微信分享面板',
74         showCancel: false
75     })
76 }
77 </script>

```

2. 场景2：订单操作菜单（状态管理）

针对不同状态的订单，动态生成操作菜单（如待付款订单显示“付款/取消”，待收货订单显示“确认收货/查看物流”），实现订单状态的灵活管理：

Code block

```

1  <template>
2    <view class="order-item">
3      <view class="order-info">
4        <text class="order-no">订单号：20240610001</text>
5        <text class="order-status" :class="status === 'pending' ? 'pending' :
'received'">
6          {{ status === 'pending' ? '待付款' : '待收货' }}

```



```
7       </text>
8     </view>
9     <button @click="showOrderMenu" class="operate-btn">操作</button>
10  </view>
11 </template>
12
13 <script setup>
14 import { ref } from 'vue'
15
16 // 订单状态: pending (待付款)、received (待收货)
17 const status = ref('pending')
18
19 // 根据订单状态生成操作菜单
20 const getOrderActionList = () => {
21   if (status.value === 'pending') {
22     return ['立即付款', '取消订单']
23   } else if (status.value === 'received') {
24     return ['确认收货', '查看物流', '申请售后']
25   }
26   return ['查看详情']
27 }
28
29 // 显示订单操作菜单
30 const showOrderMenu = () => {
31   const actionList = getOrderActionList()
32   uni.showActionSheet({
33     itemList: actionList,
34     itemColor: status.value === 'pending' ? '#1890FF' : '#52C41A',
35     success: async (res) => {
36       const action = actionList[res.tapIndex]
37       switch (action) {
38         case '立即付款':
39           await handlePay()
40           break
41         case '取消订单':
42           await cancelOrder()
43           break
44         case '确认收货':
45           await confirmReceipt()
46           break
47         case '查看物流':
48           uni.navigateTo({ url: '/pages/logistics/logistics' })
49           break
50       }
51     }
52   })
53 }
```

```
54
55 // 处理付款逻辑
56 const handlePay = async () => {
57   uni.showLoading({ title: '跳转支付中...', mask: true })
58   // 模拟支付接口请求
59   setTimeout(() => {
60     uni.hideLoading()
61     status.value = 'received' // 支付成功，更新订单状态
62     uni.showToast({ title: '支付成功', icon: 'success' })
63   }, 1500)
64 }
65
66 // 取消订单逻辑
67 const cancelOrder = async () => {
68   const modalRes = await uni.showModal({
69     title: '确认取消',
70     content: '取消订单后，订单将关闭，是否确认？'
71   })
72   if (modalRes.confirm) {
73     uni.showLoading({ title: '取消中...', mask: true })
74     // 模拟取消接口请求
75     setTimeout(() => {
76       uni.hideLoading()
77       uni.showToast({ title: '订单已取消', icon: 'success' })
78       // 后续：刷新订单列表
79     }, 1000)
80   }
81 }
82
83 // 确认收货逻辑
84 const confirmReceipt = async () => {
85   const modalRes = await uni.showModal({
86     title: '确认收货',
87     content: '请确认商品已收到，是否确认收货？'
```

五、多端适配与避坑要点

1. 多端差异与适配方案

平台	差异点	适配方案
微信小程序	1. 支持alertText参数；2. 取消按钮固定在底部；3. 操作项最多9个	1. 利用alertText添加风险提示；2. 操作项控制在6个以内
App端（iOS/Android）		

	1. 无alertText支持；2. 点击空白区域可关闭菜单；3. 样式可通过主题配置微调	1. 需添加风险提示时，结合showModal前置说明；2. 处理空白区域关闭的场景
H5端	1. 菜单样式与浏览器相关； 2. 部分浏览器不支持底部滑入动画；3. 取消按钮可能居中显示	1. 不依赖动画效果；2. 保持操作项简洁，避免复杂交互

2. 常见问题与解决方案

2.1 操作项文字过长导致显示异常？

原因： itemList中的文字超过8个字符，导致部分平台（如H5）文字换行或被截断。

解决方案：

精简操作项文字，控制在4-6个字符内（如“保存图片”而非“将图片保存到本地相册”）。必要时通过showModal在选中后补充说明，而非在菜单中显示长文本。

2.2 菜单弹出后，点击操作项无响应？

原因： 1. success回调函数未正确定义；2. 页面存在事件阻止冒泡的逻辑，阻断了菜单的点击事件。

解决方案：

检查success回调是否完整，确保函数体内无语法错误。排查菜单触发元素的事件绑定，避免使用stopPropagation()等阻止冒泡的方法。

2.3 多菜单叠加弹出？

原因： 前一个菜单未关闭时，再次调用showActionSheet，导致多个菜单叠加显示。

解决方案： 封装安全调用函数，调用前先关闭已有菜单：

Code block

```
1 // 安全调用showActionSheet的工具函数
2 const safeShowActionSheet = async (options) => {
3   // 先关闭可能存在的旧菜单
4   uni.hideActionSheet()
5   // 调用原生API并返回Promise
6   return new Promise((resolve, reject) => {
7     uni.showActionSheet({
8       ...options,
9       success: resolve,
10      fail: reject
11    })
12  })
13 }
```

```
14
15 // 使用方式
16 try {
17     const res = await safeShowActionSheet({
18         itemList: ['选项1', '选项2']
19     })
20     console.log('选中索引: ', res.tapIndex)
21 } catch (err) {
22     console.log('操作取消或失败: ', err)
23 }
```

2.4 微信小程序中alertText不显示？

原因：alertText参数仅支持微信小程序基础库2.10.0及以上版本，低版本基础库不兼容。

解决方案：

在微信开发者工具中，将基础库版本切换到2.10.0以上。兼容低版本：通过showModal在弹出菜单前显示提示文本。

六、总结

uni.showActionSheet是实现底部操作菜单的高效API，其核心价值在于原生适配多端、交互流畅且配置简单。使用时需把握以下核心原则：

- 1. 选项精简：**操作项数量控制在3-6个，文字简洁明了，避免用户选择困难。
- 2. 场景适配：**根据业务场景配置参数（如高危操作使用红色文字，必选场景隐藏取消按钮）。
- 3. 逻辑闭环：**结合showLoading、showToast等API，实现“弹出菜单→用户选择→执行逻辑→结果反馈”的完整交互。
- 4. 多端兼容：**关注各平台差异，针对小程序、App、H5的特性调整配置，避免兼容性问题。

实际开发中，建议将showActionSheet封装为通用工具函数，统一处理菜单关闭、异常捕获等逻辑，提升开发效率并保证项目交互的一致性。

（注：文档部分内容可能由 AI 生成）